Domain Overview Security Automation with AI Agents

Every category in this supplement so far has asked you to stand up a tool, point it at a target, and read what it tells you. You installed Graylog and Wazuh in **Category 3** and watched events stream into a search bar. You installed Greenbone's OpenVAS stack in **Category 5** and launched authenticated scans that returned hundreds of findings. In both cases the bottleneck was never the tool — it was you. A home lab generates a trickle of alerts and a few dozen vulnerabilities, and that is already enough to bury an analyst in triage. A real environment generates that volume every few minutes.

Security automation is the discipline of moving the repetitive, judgment-light parts of that triage off the analyst's plate. Historically this meant rule engines and SOAR platforms — StackStorm, Shuffle, TheHive — where a human encoded every decision in advance as "if this alert, then that action." Those systems are powerful but brittle: they only handle the cases someone anticipated, and the rules rot as the environment changes.

Large language models change the shape of the problem. Instead of pre-encoding every decision, you give a model the raw signal — a log event, a scan finding — plus context about your environment and a clear instruction, and it produces a structured judgment: severity, plain-language explanation, likely cause, and recommended next step. This is what people mean by an **AI agent** in a security context: a program that loops over incoming data, calls a model to reason about each item, and emits an enriched, prioritized result a human can act on quickly.

> **A note on what "agent" means here**
>
> The word **agent** is overloaded. In this lab an agent is a Python program you write that (1) pulls data from a source, (2) sends it to the Claude API with a carefully written prompt, (3) parses the structured response, and (4) writes the result somewhere useful. It does **not** take autonomous action against your systems — it reads, reasons, and reports. That read-only, human-in-the-loop posture is a deliberate safety choice we return to throughout.

This category uses Anthropic's Claude models through their HTTP API. You will write the agents in Python because the SDK is mature and the same patterns transfer directly to any other language. The model never touches your machines; it only sees the text you choose to send it, and it only ever returns text. Every action against your lab — reading logs, pulling scan reports — is code **you** wrote and control.

Why pair an LLM with a SIEM and a vulnerability scanner?

These two data sources are the canonical examples of "high volume, high tedium, high judgment" security work, and they sit at opposite ends of a useful spectrum:

- **SIEM alerts are fast, noisy, and ephemeral.** A Wazuh or Graylog alert arrives with a rule ID, a raw log line, and a numeric level. Deciding whether it matters requires correlating it against

what is normal for the host, the time of day, and recent activity. Most are benign; a few are not. The triage judgment is qualitative.

- **Vulnerability findings are slow, dense, and overwhelming.** An OpenVAS report lists every weakness on every host with a CVSS score and a technical description. The judgment here is prioritization: which of these 200 findings actually threaten **this** environment, given what is internet-facing, what holds sensitive data, and what is already mitigated?

An LLM is well suited to both because both are fundamentally reading-comprehension-plus-prioritization tasks expressed in natural language and semi-structured data — exactly what these models do well. By the end of this category you will have an agent that turns a stream of SIEM alerts into a ranked, explained queue, and a second agent that turns a raw scan report into a remediation plan written for a human. The third lab wires them together.

What you will build

Lab	Agent	Reads from	Produces
1	**Log Triage Agent**	Wazuh / Graylog REST API	Ranked, explained alert queue (JSON + Markdown)
2	**Vuln Report Agent**	Greenbone GMP (python-gvm)	Prioritized remediation plan per host
3	**Triage Orchestrator**	Both of the above	Unified daily security brief with cross-correlation

Table 1 – AI lab breakdoiwn

Cost and safety guardrails (read before Lab 1)

- Every API call costs money. The labs are designed to process small batches (10–50 items) so a full run costs cents, not dollars. We set explicit limits in code.
- Never send secrets to the API. The agents are written to strip credentials, tokens, and private keys from any data before it leaves your machine.
- The agents are read-only. They never write to Wazuh, never re-run scans, never touch a host. A human reads the output and decides what to do.
- Treat the input as hostile and the output as untrusted. A log line or scan banner can carry text crafted to steer the model — a prompt injection — and a SIEM is exactly where an attacker can plant one. The read-only design is what contains this: the worst case is a wrong verdict a human reviews, never an action. Validate the fields you parse (e.g. that severity is one of the allowed values) and never execute or act on the model's text directly.

Technology Overview

The Claude API

What is it	A web (HTTP) service from Anthropic that accepts a list of messages and returns a model-generated reply. You call it over HTTPS with an API key; the official `anthropic` Python package wraps it.
Why use it	It performs the reading-comprehension and prioritization judgment at the heart of triage without you having to encode rules. It returns structured JSON on demand, which makes its output easy for the rest of your program to consume.
What it provides	A single endpoint — the **Messages API** — that takes a model name, a system prompt, a list of user/assistant turns, and a `max_tokens` cap, and returns text and/or tool-use blocks.

Table 2 – Claude api breakdown

Models and identifiers used in this lab (current as of mid-2026 — re-check the lineup before a run)

- **claude-haiku-4-5-20251001** — fastest and cheapest. We use it for high-volume, low-complexity triage (Lab 1).
- **claude-sonnet-4-6** — balanced. We use it for the denser reasoning in the vulnerability and orchestration labs.
- **claude-opus-4-8** — most capable. Optional upgrade for the orchestrator if you want the richest cross-correlation. Model IDs are pinned snapshots; the dateless 4.6-and-later IDs (for example claude-sonnet-4-6, claude-opus-4-8) are themselves fixed snapshots, not moving pointers.

Greenbone / OpenVAS via python-gvm

What is it	Greenbone Community Edition (the open-source OpenVAS stack you installed in Category 5) exposes the **Greenbone Management Protocol (GMP)** over a local socket. The `python-gvm` library speaks GMP from Python.

Why use it	It lets your agent pull finished scan reports programmatically — every result, host, CVSS score, and description — instead of you exporting PDFs by hand.
What it provides	Authenticated, read-only access to tasks, targets, and reports. Lab 2 uses it only to **read** the latest report for a task and hand the findings to the agent.

Table 3 – Greenbone breakdown

Wazuh and Graylog APIs

Both SIEM tools from Category 3 expose REST APIs. **Wazuh** stores fired alerts in its indexer (an OpenSearch instance, default port 9200); the wazuh-alerts-* index returns each event with its rule level, description, and the agent it came from. (The manager API on port 55000 serves the static ruleset, not fired events, so Lab 1 reads from the indexer.) **Graylog** provides a REST API (default port 9000) with a search endpoint that returns messages matching a query over a time range. Lab 1 includes paths for both; pick whichever you deployed.

Lab Topology

All three labs run from a single Debian 13 workstation — the same machine you have used throughout the supplement, or a fresh VM with network reach to your Category 3 and Category 5 hosts. The agent code lives here; it makes outbound HTTPS calls to the Claude API and inbound-LAN calls to your SIEM and scanner.

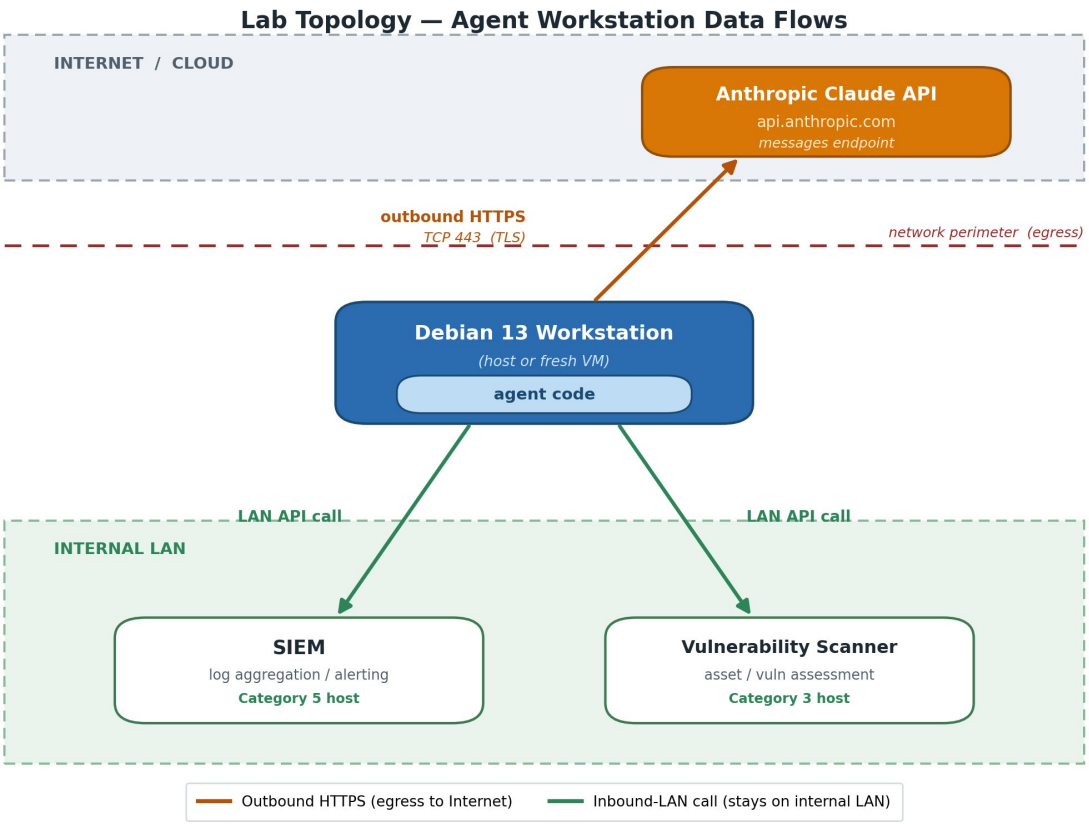

Figure 1 – AI Agent Lab Topology

Network direction matters

The only traffic leaving your network is the text you send to the Claude API over TLS. Traffic to the SIEM and scanner stays on your LAN. If your lab is air-gapped, the agents cannot reach the API — you would need a model running locally, which is outside this lab's scope but noted in "Where to Go Next."

Prerequisites

- **A Debian 13 host** — physical or virtual. Recommended: 2 vCPU, 4 GB RAM, 40 GB disk.
- **Sudo access** — for installation of prerequisites.
- **Internet access** — Outbound HTTPS (TCP 443) from this host to PyPI and to the Claude API, so the pip install and the Step 3 smoke test both succeed.
- **Anthropic Console account** — with billing enabled. A key on an unfunded account authenticates but every call returns a credit error.

Shared Setup: Python Environment and API Key

Install with Python3, the venv and pip packages are not present by default, so install them first. (Without python3-venv, the venv step fails.) Complete this once. All three labs reuse the same virtual environment and key.

Step 1 – Setup AI shared system

1. Setup system and install prerequisites:

```
sudo apt install python3-venv python3-pip
```

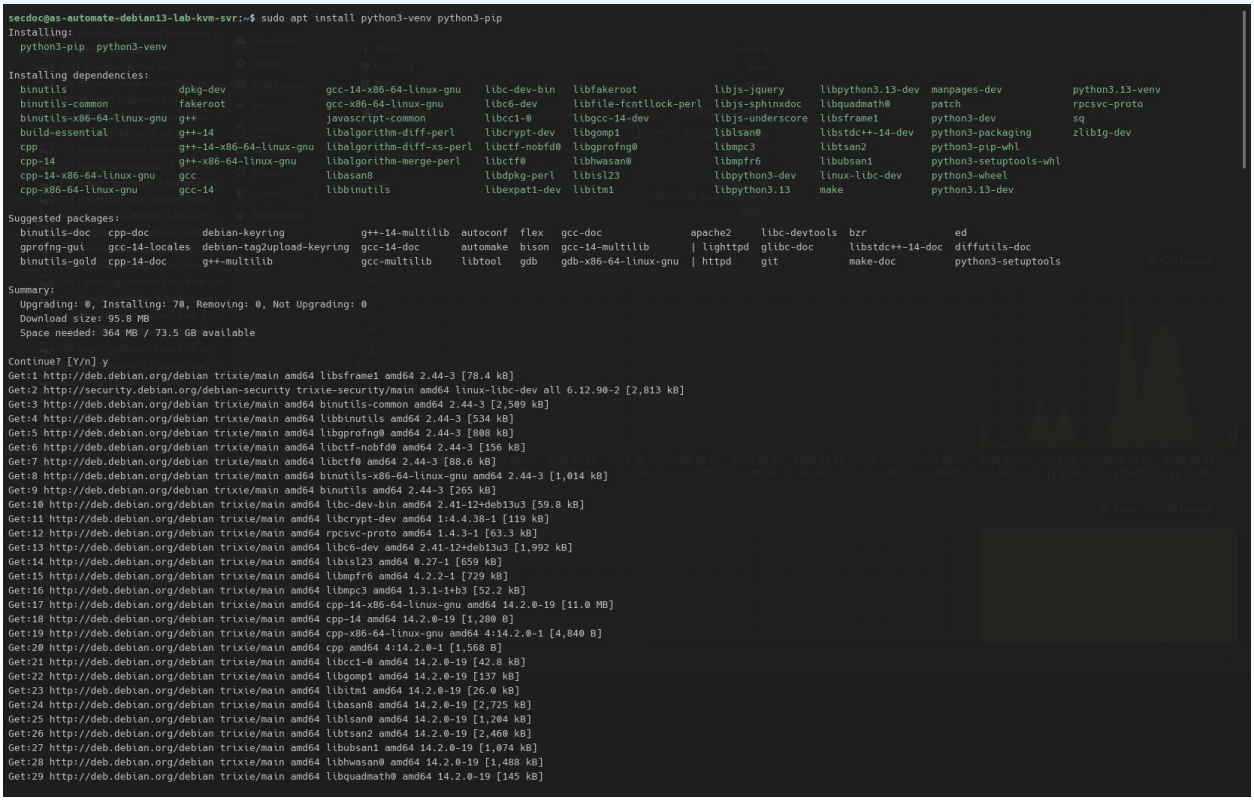

Figure 2 - Install prerequisites

2. Create the project and virtual environment

```
mkdir -p ~/cah2-ai-agents && cd ~/cah2-ai-agents
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install "anthropic>=0.40" "python-gvm>=27" requests
```

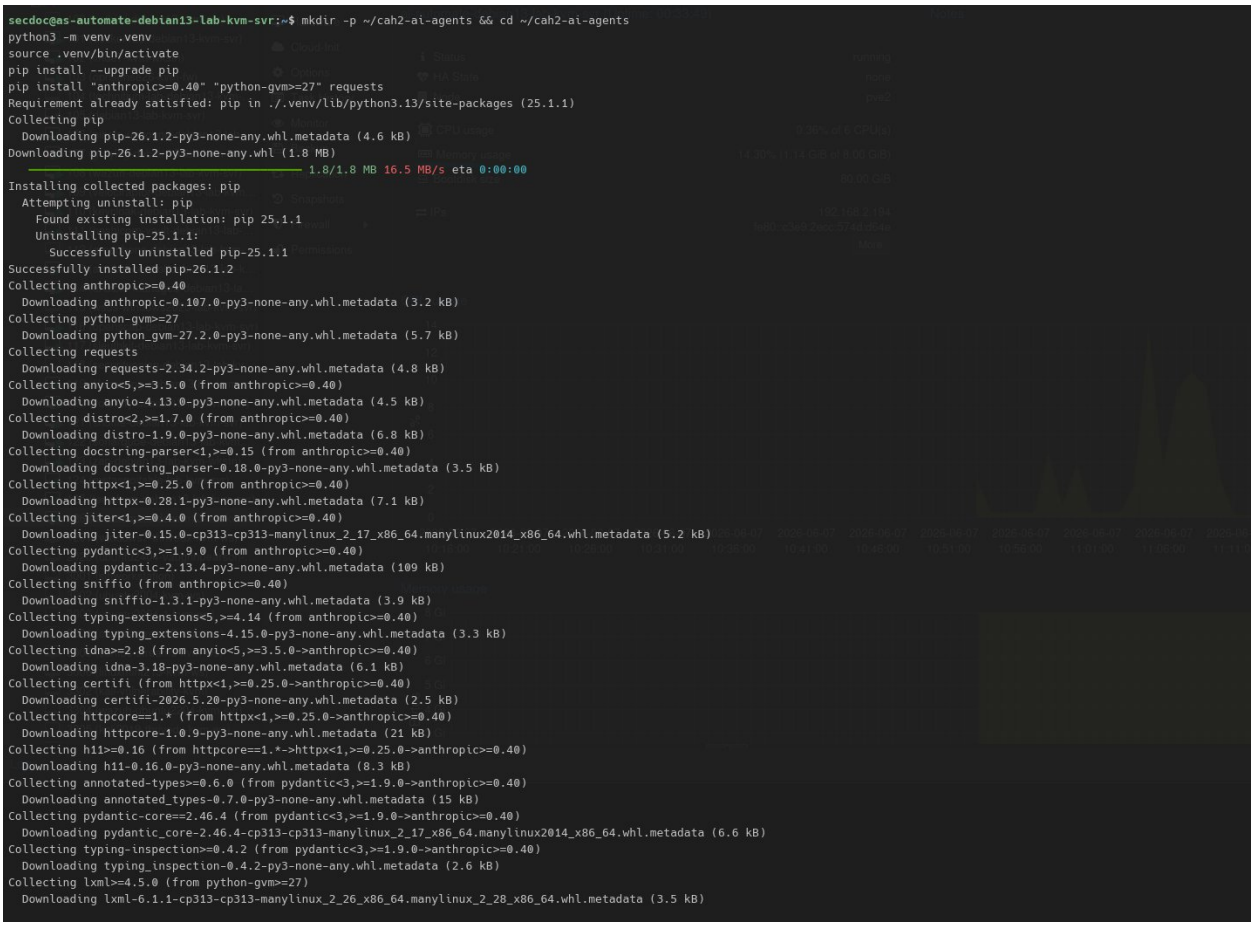

Figure 3 - Create agents directory and install pip components

Step 2 — Obtain and store an API key

1. Create a key in the Anthropic Console (console.anthropic.com) under **API Keys**. Treat it like a password. Store it in an environment file the agents will read, and make sure that file is never committed to version control.

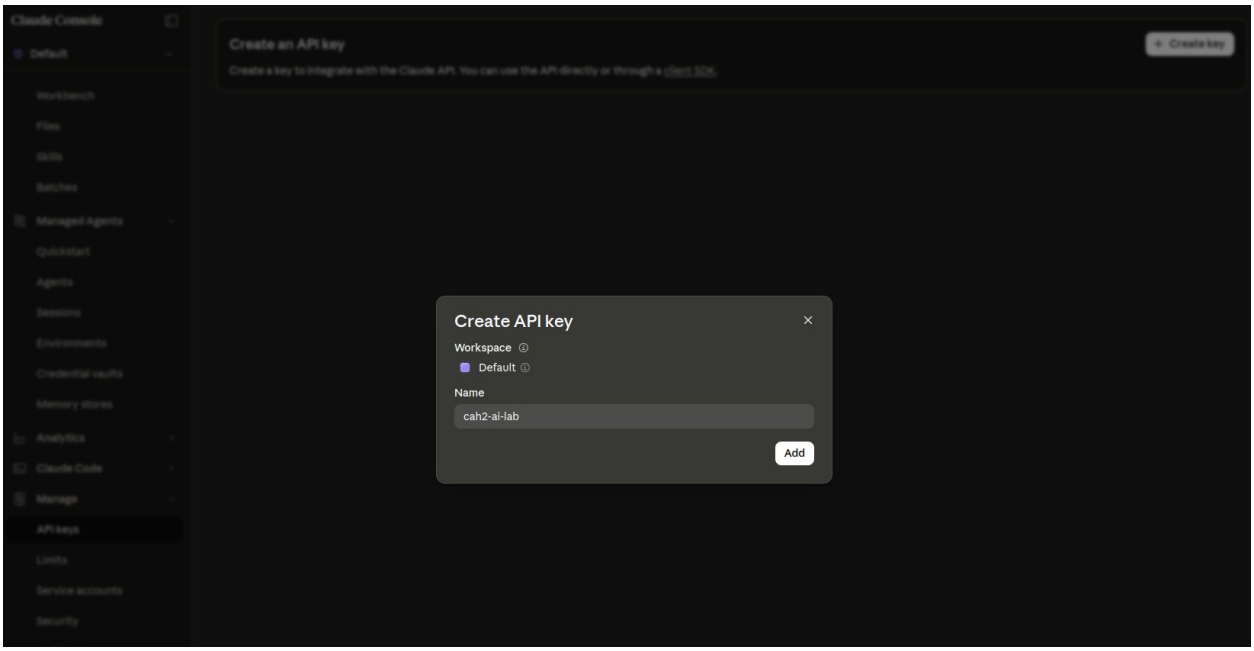

Figure 4 - Create api key in claude console

```
# Store the key OUTSIDE your code, readable only by you
echo 'export ANTHROPIC_API_KEY="sk-ant-...your-key..."' > ~/.cah2-secrets
chmod 600 ~/.cah2-secrets
source ~/.cah2-secrets

# Confirm it is set (prints the first 12 chars only)
echo "${ANTHROPIC_API_KEY:0:12}..."
```

Figure 5 – Store api key and verify

Why an env var and not a config file?

A key pasted into a script eventually ends up in a screenshot, a paste, or a git history. Reading it from the environment keeps it out of every file the agent touches. The chmod 600 ensures only your user can read the secrets file.

Step 3 — Verify connectivity with a one-line smoke test

1. Now verify connectivity by running the following script:

```
python3 - <<'PY'
import anthropic
client = anthropic.Anthropic()  # reads ANTHROPIC_API_KEY from env
msg = client.messages.create(
    model="claude-haiku-4-5-20251001",
    max_tokens=64,
    messages=[{"role": "user", "content": "Reply with the single word: ready"}],
)
print(msg.content[0].text)
PY
```

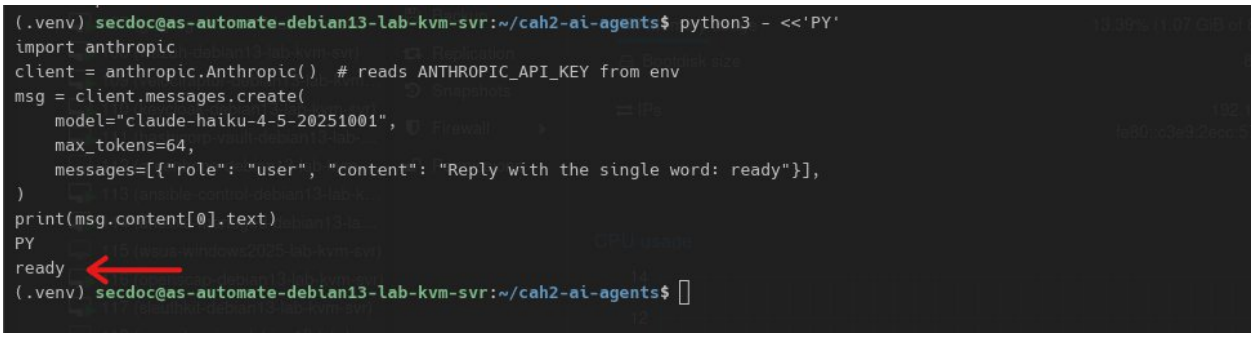

Figure 6 – Smoke test validation

If you see ready, your environment, key, and network path to the API are all working. If you get an authentication error, re-check Step 2; if you get a connection error, check that outbound 443 is permitted from this host.

Validate the Lab

- Virtual environment created and activated
- anthropic, python-gvm, and requests installed
- API key stored in a 600-permission secrets file, not in code
- Smoke test prints ready

Lab: Building a SIEM Log Triage Agent

In this lab you build an agent that reads recent alerts from your Category 3 SIEM, asks Claude to triage each one, and produces a ranked queue with a one-line explanation and a recommended action for every alert. You will start with a single hard-coded alert to learn the prompt, then wire it to the live API.

Objectives

- Pull a batch of recent alerts from Wazuh or Graylog via its REST API.
- Design a system prompt that makes Claude return strict, parseable JSON.
- Process a batch, sort by the model's severity judgment, and write a human-readable brief.
- Strip sensitive fields before any data leaves the host.

Prerequisites

- A working Wazuh **or** Graylog instance from Category 3 with at least a few alerts present. (Generate some by running an Nmap scan against a monitored host, or a few failed SSH logins.)
- The shared setup completed and the virtual environment active.

Part 1 — Design the triage prompt

1. The quality of an agent is mostly the quality of its prompt. We want Claude to act as a tier-1 SOC analyst and return a fixed JSON shape so the rest of the program can rely on it. Create triage_prompt.py:

```
# triage_prompt.py
SYSTEM_PROMPT = """You are a tier-1 SOC analyst triaging SIEM alerts in a
small enterprise/home-lab environment. For each alert you receive, judge how
urgently a human should look at it.

Respond with ONLY a JSON object (no prose, no markdown fences) of the form:
{
  "severity": "critical" | "high" | "medium" | "low" | "info",
  "confidence": 0.0-1.0,
  "summary": "one sentence, plain English, what happened",
  "likely_cause": "most probable benign and malicious explanation",
  "recommended_action": "the single next step a human should take",
  "false_positive_likelihood": "high" | "medium" | "low"
}

Be calibrated: most alerts in a lab are benign. Reserve 'critical' and 'high'
for genuine indicators of compromise. If the alert lacks context to judge,
say so in 'summary' and set confidence accordingly."""
```

Prompt-engineering notes

- We pin the **output schema** explicitly. Asking for "only JSON, no markdown fences" is the single most important line for making the output machine-parseable.
- We tell the model the **environment is small** and "most alerts are benign," which calibrates it away from crying wolf on every event.
- We ask for both a benign **and** a malicious explanation, which surfaces the model's reasoning and helps a human sanity-check it.
- If you want to understand more about AI coding, prompt engineering and so on, check out the many great books by the authors at https://www.packt.com/.

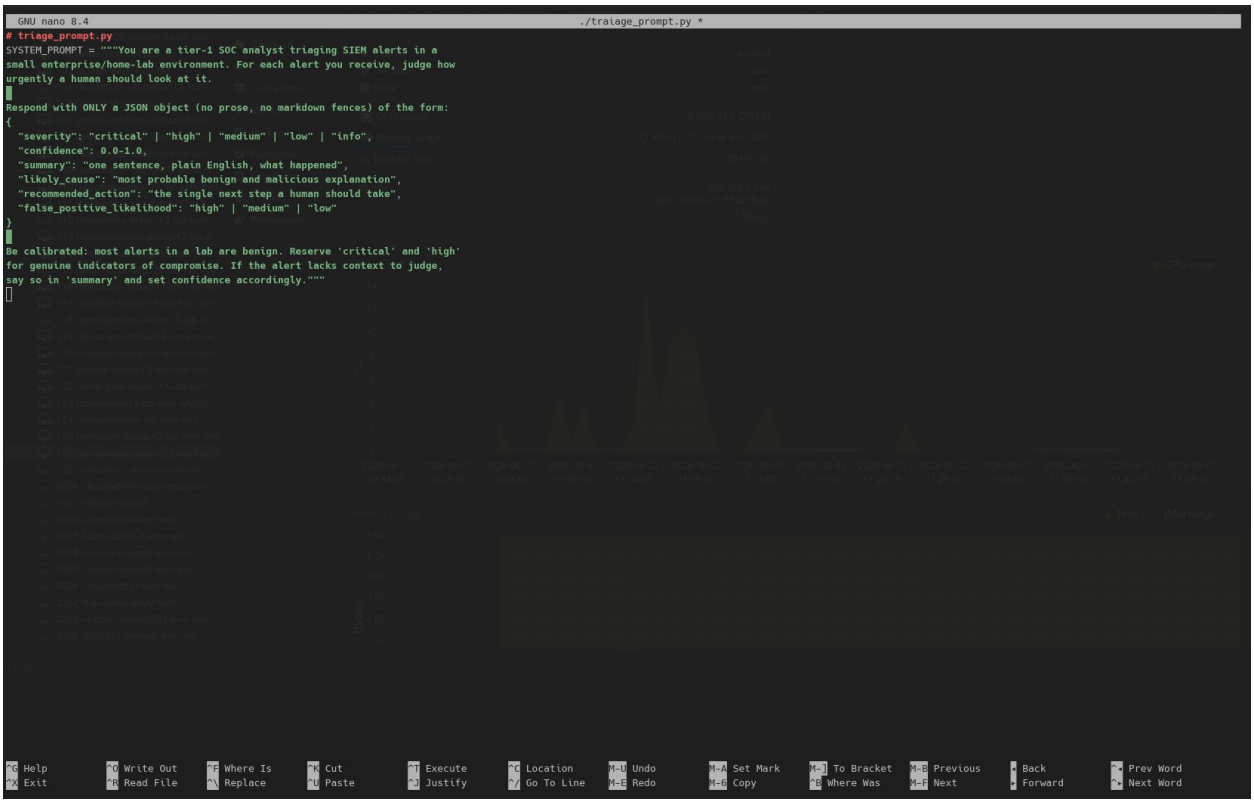

Figure 7 – Create triage prompt python code

Part 2 — Pull alerts from the SIEM

1. Create siem_source.py. It exposes one function, fetch_alerts(limit), returning a list of plain dicts. Use whichever block matches your deployment; the rest of the lab is identical either way.

Option 1 — Wazuh

```
# siem_source.py  (Wazuh variant)
```

```
import os, requests, urllib3
urllib3.disable_warnings()  # lab uses a self-signed cert

# Fired alerts live in the Wazuh indexer (OpenSearch), NOT the manager API.
# The manager /rules endpoint returns the static ruleset, not events.
WAZUH = os.environ.get("WAZUH_INDEXER_URL", "https://127.0.0.1:9200")
WUSER = os.environ.get("WAZUH_USER", "admin")
WPASS = os.environ["WAZUH_PASS"]

def fetch_alerts(limit=20):
    # Pull the most recent fired alerts from the wazuh-alerts-* index.
    body = {
        "size": limit,
        "sort": [{"timestamp": {"order": "desc"}}],
        "query": {"match_all": {}},
    }
    r = requests.get(f"{WAZUH}/wazuh-alerts-*/_search",
                     json=body, auth=(WUSER, WPASS),
                     verify=False, timeout=15)
    r.raise_for_status()
    out = []
    for hit in r.json()["hits"]["hits"]:
        src = hit["_source"]
        rule = src.get("rule", {})
        agent = src.get("agent", {})
        out.append({
            "id": rule.get("id"),
            "level": rule.get("level"),
            "description": rule.get("description"),
            "groups": ",".join(rule.get("groups", [])),
            "host": agent.get("name") or agent.get("ip") or "unknown",
            "full_log": (src.get("full_log") or "")[:500],
        })
    return out
```

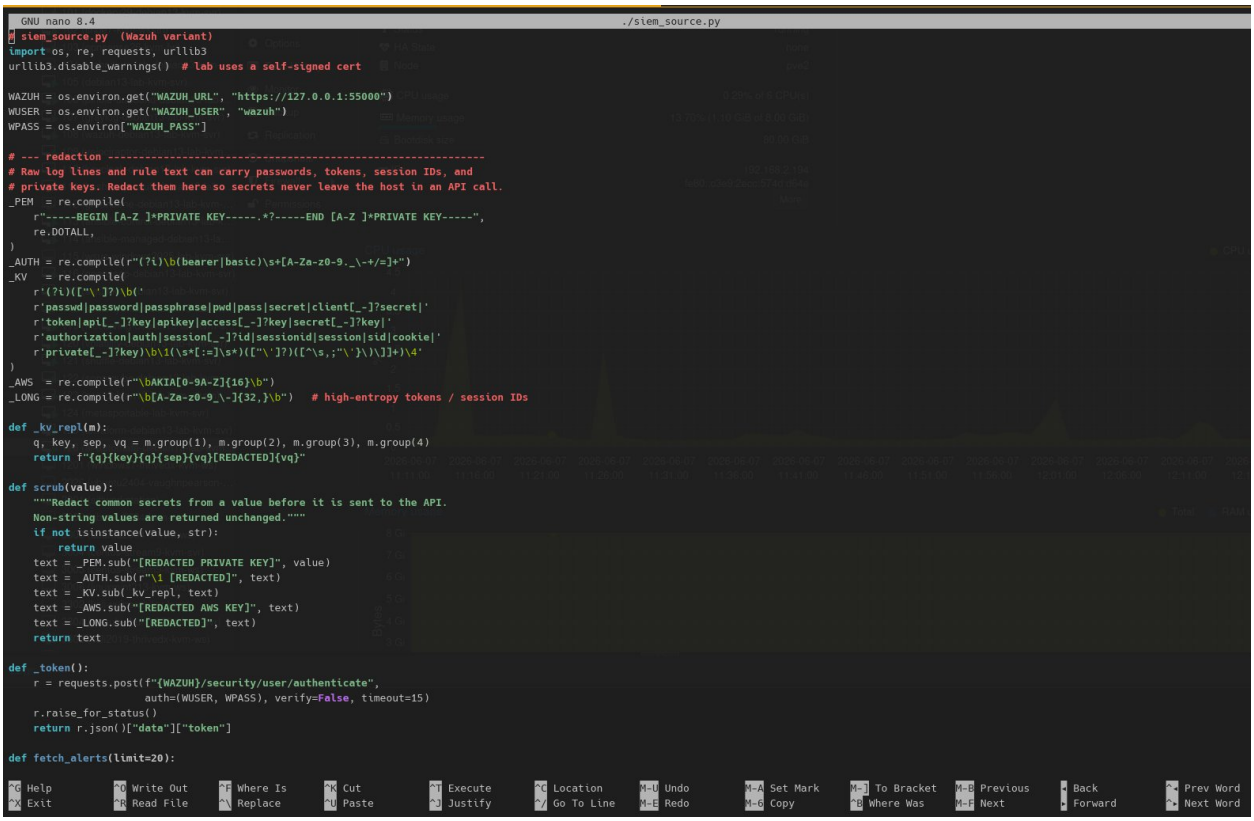

Figure 8 – Wazuh python script

Option 2 — Graylog

```
# siem_source.py  (Graylog variant)
# NOTE: /search/universal/relative is the simple "legacy" search endpoint.
# It still works in current Graylog but is deprecated; if it is removed on
# your version, switch to the Views Search API (POST /api/views/search).
import os, requests

GRAYLOG = os.environ.get("GRAYLOG_URL", "http://127.0.0.1:9000")
GTOKEN  = os.environ["GRAYLOG_TOKEN"]  # create under System > Users > API tokens

def fetch_alerts(limit=20):
    params = {"query": "*", "range": 3600, "limit": limit, "sort": "timestamp:desc"}
    r = requests.get(f"{GRAYLOG}/api/search/universal/relative",
                     params=params, auth=(GTOKEN, "token"),
                     headers={"Accept": "application/json"}, timeout=20)
    r.raise_for_status()
```

```
    out = []
    for m in r.json().get("messages", []):
        msg = m["message"]
        out.append({
            "id": msg.get("_id"),
            "source": msg.get("source"),
            "message": msg.get("message"),
            "timestamp": msg.get("timestamp"),
        })
    return out
```

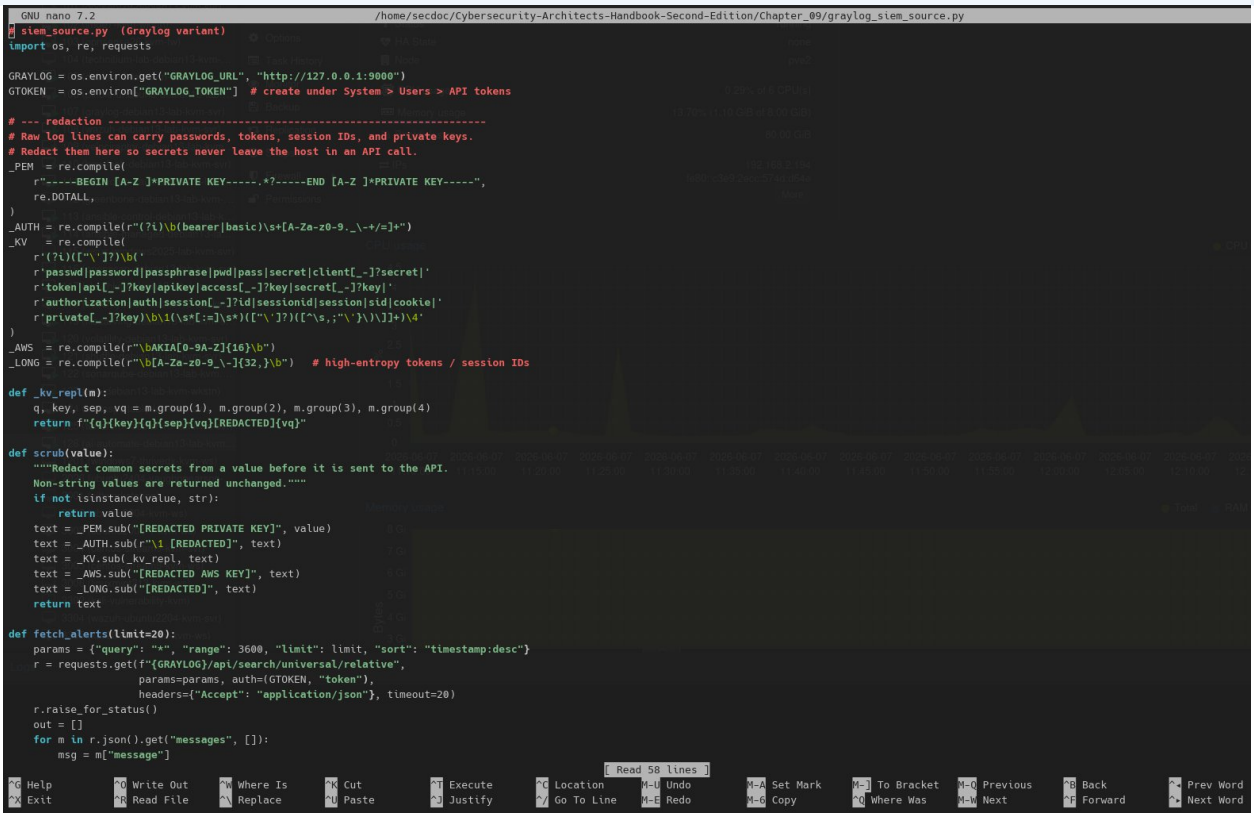

Figure 9 – Graylog python script

Strip secrets before they leave the host

Whichever source you use, raw log lines can contain tokens, session IDs, or passwords. Add a scrub step so they never reach the API. A simple redaction pass covers the common cases:

```
# add to siem_source.py
```

```
import re

_PATTERNS = [

(re.compile(r"(?i)(password|passwd|pwd|secret|token|api[_-]?key)\s*[=:]\s*\S+"
), r"\1=[REDACTED]"),
    (re.compile(r"(?i)bearer\s+[A-Za-z0-9._-]+"), "bearer [REDACTED]"),
    (re.compile(r"\b[A-Za-z0-9]{32,}\b"), "[REDACTED-LONG-TOKEN]"),
]
def scrub(text):
    if not isinstance(text, str):
        return text
    for pat, repl in _PATTERNS:
        text = pat.sub(repl, text)
    return text
```

```
# --- redaction -------------------------------------------------------------
# Raw log lines and rule text can carry passwords, tokens, session IDs, and
# private keys. Redact them here so secrets never leave the host in an API call.
_PEM  = re.compile(
    r"-----BEGIN [A-Z ]*PRIVATE KEY-----.*?-----END [A-Z ]*PRIVATE KEY-----",
    re.DOTALL,
)
_AUTH = re.compile(r"(?i)\b(bearer|basic)\s+[A-Za-z0-9._\-+/=]+")
_KV   = re.compile(
    r'(?i)(["\']?)\b('
    r'passwd|password|passphrase|pwd|pass|secret|client[_-]?secret|'
    r'token|api[_-]?key|apikey|access[_-]?key|secret[_-]?key|'
    r'authorization|auth|session[_-]?id|sessionid|session|sid|cookie|'
    r'private[_-]?key)\b\1(\s*[:=]\s*)(["\']?)([^\s,;"\'}\)\]]+)\4'
)
_AWS  = re.compile(r"\bAKIA[0-9A-Z]{16}\b")
_LONG = re.compile(r"\b[A-Za-z0-9_\-]{32,}\b")   # high-entropy tokens / session IDs

def _kv_repl(m):
    q, key, sep, vq = m.group(1), m.group(2), m.group(3), m.group(4)
    return f"{q}{key}{q}{sep}{vq}[REDACTED]{vq}"

def scrub(value):
    """Redact common secrets from a value before it is sent to the API.
    Non-string values are returned unchanged."""
    if not isinstance(value, str):
        return value
    text = _PEM.sub("[REDACTED PRIVATE KEY]", value)
    text = _AUTH.sub(r"\1 [REDACTED]", text)
    text = _KV.sub(_kv_repl, text)
    text = _AWS.sub("[REDACTED AWS KEY]", text)
    text = _LONG.sub("[REDACTED]", text)
    return text
```

Figure 10 – Secrets redaction

Part 3 — Wire the agent together

1. Create triage_agent.py. This is the core loop: fetch, scrub, ask Claude, parse, rank, report.

```
# triage_agent.py
import json, sys
import anthropic
from triage_prompt import SYSTEM_PROMPT
from siem_source import fetch_alerts, scrub

MODEL = "claude-haiku-4-5-20251001"   # fast + cheap for high-volume triage
RANK  = {"critical": 0, "high": 1, "medium": 2, "low": 3, "info": 4}

client = anthropic.Anthropic()

def triage_one(alert):
    clean = {k: scrub(v) for k, v in alert.items()}
    resp = client.messages.create(
        model=MODEL,
        max_tokens=400,
        system=SYSTEM_PROMPT,
        messages=[{"role": "user",
                   "content": "Triage this alert:\n" + json.dumps(clean,
indent=2)}],
    )
    raw = resp.content[0].text.strip()
    try:
        verdict = json.loads(raw)
    except json.JSONDecodeError:
        # model wrapped JSON in fences or prose; recover the object
        start, end = raw.find("{"), raw.rfind("}")
        verdict = json.loads(raw[start:end + 1])
    verdict["_alert"] = clean
    return verdict

def main(limit=15):
    alerts = fetch_alerts(limit)
    print(f"Fetched {len(alerts)} alerts; triaging...", file=sys.stderr)
    results = []
```

```
    for a in alerts:
        try:
            results.append(triage_one(a))
        except Exception as e:
            print(f"  skip one alert: {e}", file=sys.stderr)
    results.sort(key=lambda v: RANK.get(v.get("severity", "info"), 9))
    return results


if __name__ == "__main__":
    lim = int(sys.argv[1]) if len(sys.argv) > 1 else 15
    out = main(lim)
    with open("triage_results.json", "w") as f:
        json.dump(out, f, indent=2)
    print(f"Wrote triage_results.json with {len(out)} verdicts")
```

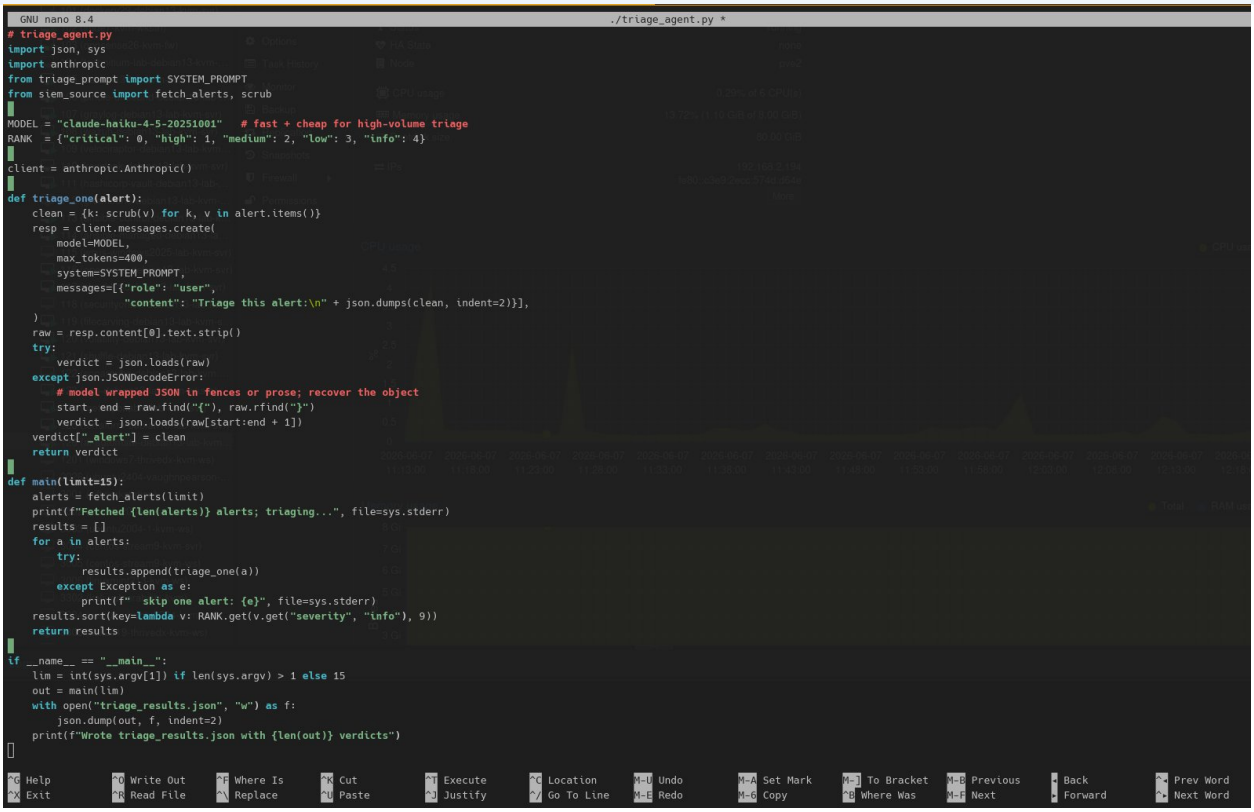

Figure 11 – Triage agent python code

2. Run it:

```
# (Wazuh) export creds first, or (Graylog) export the token
```

```
export WAZUH_PASS="...your-wazuh-indexer-password..."  # the indexer (admin) password

python triage_agent.py 15
```

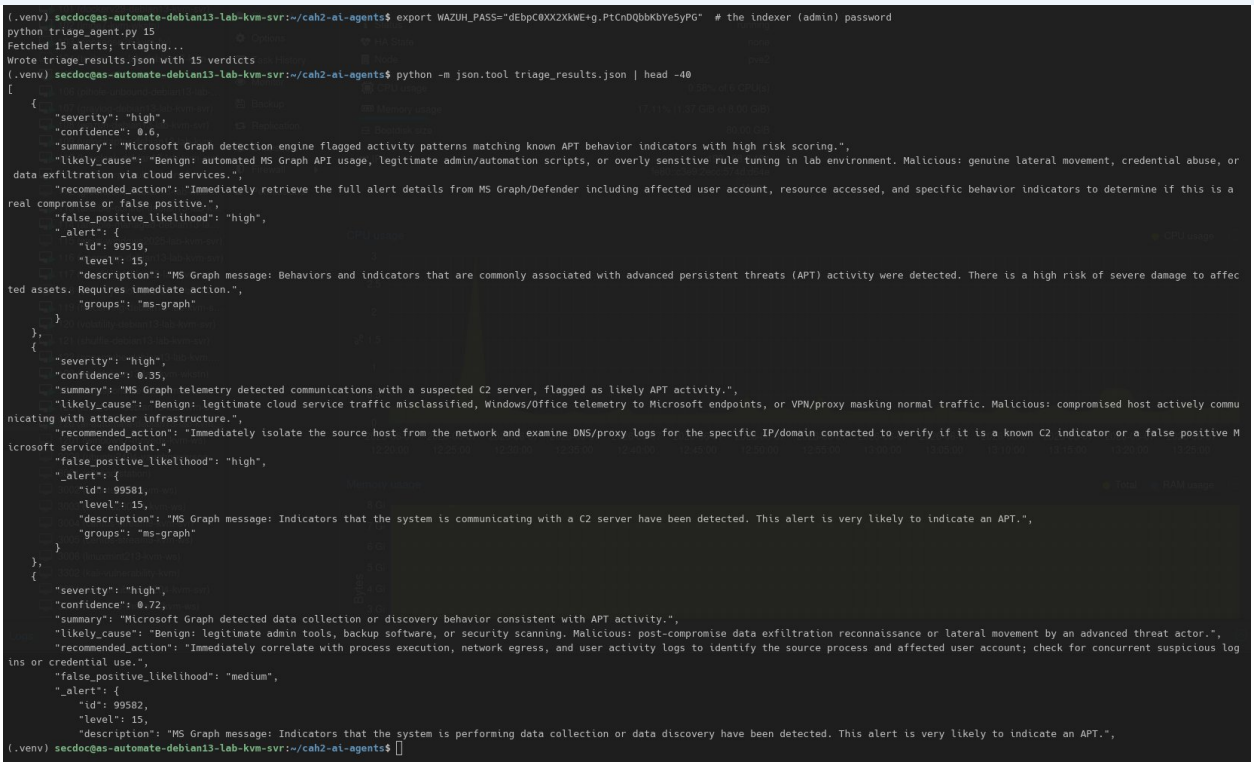

Figure 12 – Successful alert triage

Part 4 — Render a human-readable brief

1. JSON is for machines. Create render_brief.py to turn the ranked results into a Markdown brief an analyst can skim in thirty seconds.

```
# render_brief.py
import json, datetime


with open("triage_results.json") as f:
    results = json.load(f)


lines = [f"# SIEM Triage Brief — {datetime.date.today()}", ""]
counts = {}
for r in results:
    counts[r["severity"]] = counts.get(r["severity"], 0) + 1
lines.append("**Summary:** " + ", ".join(f"{v} {k}" for k, v in counts.items()))
```

```
lines.append("")

for r in results:

    lines.append(f"## [{r['severity'].upper()}] {r['summary']}")

    lines.append(f"- **Confidence:** {r['confidence']}  |  **FP likelihood:** {r[\"false_positive_likelihood\"]}")

    lines.append(f"- **Likely cause:** {r['likely_cause']}")

    lines.append(f"- **Recommended action:** {r['recommended_action']}")

    lines.append("")

open("triage_brief.md", "w").write("\n".join(lines))

print("Wrote triage_brief.md")
```

Run python render_brief.py and open triage_brief.md. You now have a prioritized, explained queue — critical items at the top, each with the model's reasoning and a concrete next step.

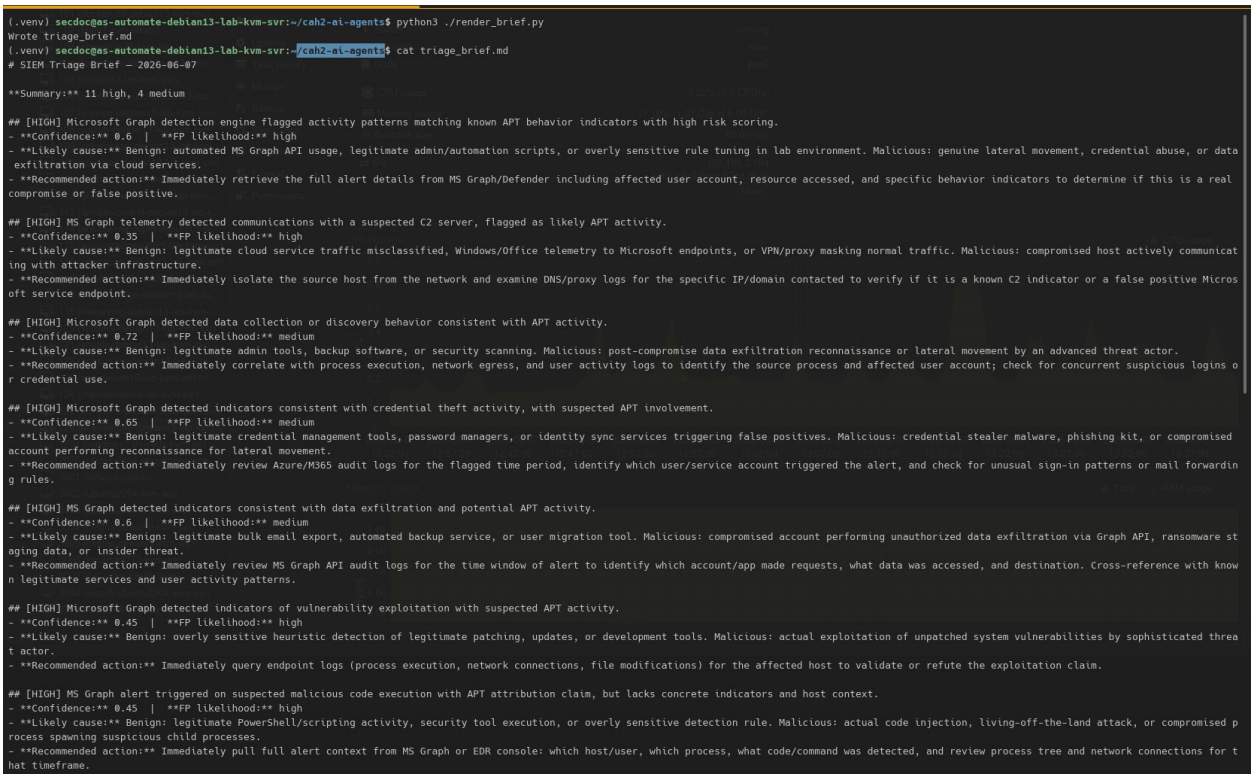

Figure 13 – Markdown report

Validate the Lab

- triage_agent.py 15 runs without error and writes triage_results.json
- Every verdict in the JSON has all six required fields
- Results are sorted with the most severe first
- A log line containing a fake password=... appears [REDACTED] in the JSON
- triage_brief.md renders a readable summary with severity counts at the top

- Re-running on the same alerts produces stable severity judgments (calibration sanity check)

Troubleshooting

Symptom	Likely cause and fix
JSONDecodeError	The model added prose or fences. The recovery branch in triage_one handles most cases; if it persists, tighten the system prompt's "only JSON" instruction.
Authentication error from SIEM	Wrong creds or expired token. For Wazuh, confirm the API password; for Graylog, regenerate the API token under System > Users.
Every alert comes back "low"	Expected in a quiet lab. Generate real signal: run an Nmap scan or several failed SSH logins against a monitored host, then re-run.
API rate-limit (429)	You are sending too fast. Lower the batch size, or add a short sleep between calls. Tier-1 keys have modest per-minute limits.

Table 4 - Troubleshooting

Lab: Building a Vulnerability Report Agent

A finished OpenVAS scan is a wall of findings. This agent pulls the latest report for a task via GMP, groups findings by host, and asks Claude to produce a **prioritized remediation plan** — not a restatement of CVSS scores, but a human-facing answer to "what do I fix first, and why does it matter **here**?"

Objectives

- Connect to Greenbone over GMP with python-gvm and pull the latest report for a task.
- Reduce raw findings to a compact, token-efficient structure.
- Prompt Claude to prioritize by real-world risk in context, not raw score.
- Emit a per-host remediation plan ordered by what to fix first.

Prerequisites

- A Greenbone Community Edition install from Category 5 with at least one **completed** scan task.
- GMP credentials (the `admin` user you created during the Category 5 setup).
- The GMP socket path — typically `/run/gvmd/gvmd.sock` — or a TLS connection if you enabled it. Note that this socket is local to the Greenbone host: a Unix socket cannot be reached over the network. If the agent runs on a separate machine — the usual setup, where the agent box and

the scanner are different hosts — connect by forwarding the socket over SSH (then keep the local Unix-socket path) or by setting GVM_CONN=tls or ssh, as shown in Running the agent from a separate host below.

Part 1 — Pull a report over GMP

1. Create gvm_source.py. This connects over the local Unix socket, authenticates, finds the most recent report, and flattens it into a list of findings.

```
# gvm_source.py
import os
from gvm.connections import (
    UnixSocketConnection,
    TLSConnection,
    SSHConnection,
)
from gvm.protocols.gmp import Gmp
from gvm.transforms import EtreeCheckCommandTransform

# Connection selection via GVM_CONN:
#   unix (default) - local gvmd socket, OR a remote socket forwarded here
#                    over SSH (set GVMD_SOCKET to the forwarded path).
#   tls            - network TLS to gvmd on GVM_HOST:GVM_PORT (9390). gvmd
#                    must be configured to expose a TLS listener.
#   ssh            - python-gvm SSH transport to a gvmd host.
SOCK  = os.environ.get("GVMD_SOCKET", "/run/gvmd/gvmd.sock")
GUSER = os.environ.get("GVM_USER", "admin")
GPASS = os.environ["GVM_PASS"]

def _connection():
    mode = os.environ.get("GVM_CONN", "unix").lower()
    if mode == "tls":
        return TLSConnection(
            hostname=os.environ["GVM_HOST"],
            port=int(os.environ.get("GVM_PORT", "9390")),
        )
    if mode == "ssh":
```

```
        return SSHConnection(
            hostname=os.environ["GVM_HOST"],
            port=int(os.environ.get("GVM_SSH_PORT", "22")),
            username=os.environ["GVM_SSH_USER"],
            password=os.environ.get("GVM_SSH_PASS"),
        )
    # default: local Unix socket (or an SSH-forwarded one)
    return UnixSocketConnection(path=SOCK)

def latest_report_findings(min_qod=70):
    conn = _connection()
    transform = EtreeCheckCommandTransform()
    findings = []
    with Gmp(connection=conn, transform=transform) as gmp:
        gmp.authenticate(GUSER, GPASS)
        # newest report first
        reports = gmp.get_reports(filter_string="sort-reverse=date rows=1")
        report_id = reports.find("report").get("id")
        full = gmp.get_report(report_id, details=True)
        for res in full.iter("result"):
            qod = res.findtext("qod/value") or "0"
            if int(qod) < min_qod:
                continue   # skip low quality-of-detection noise
            findings.append({
                "host": res.findtext("host") or "unknown",
                "name": (res.findtext("name") or "").strip(),
                "severity": res.findtext("severity") or "0.0",
                "port": res.findtext("port") or "",
                # truncate the long description to control token use
                "summary": (res.findtext("description") or "")[:600],
            })
    return findings
```

Why filter on QoD?

Greenbone attaches a **Quality of Detection** score to every result — how confident the scanner is that the finding is real. Filtering to QoD ≥ 70 strips the speculative findings that would otherwise dilute the agent's plan and waste tokens. This mirrors the manual triage you would do in the Greenbone web UI.

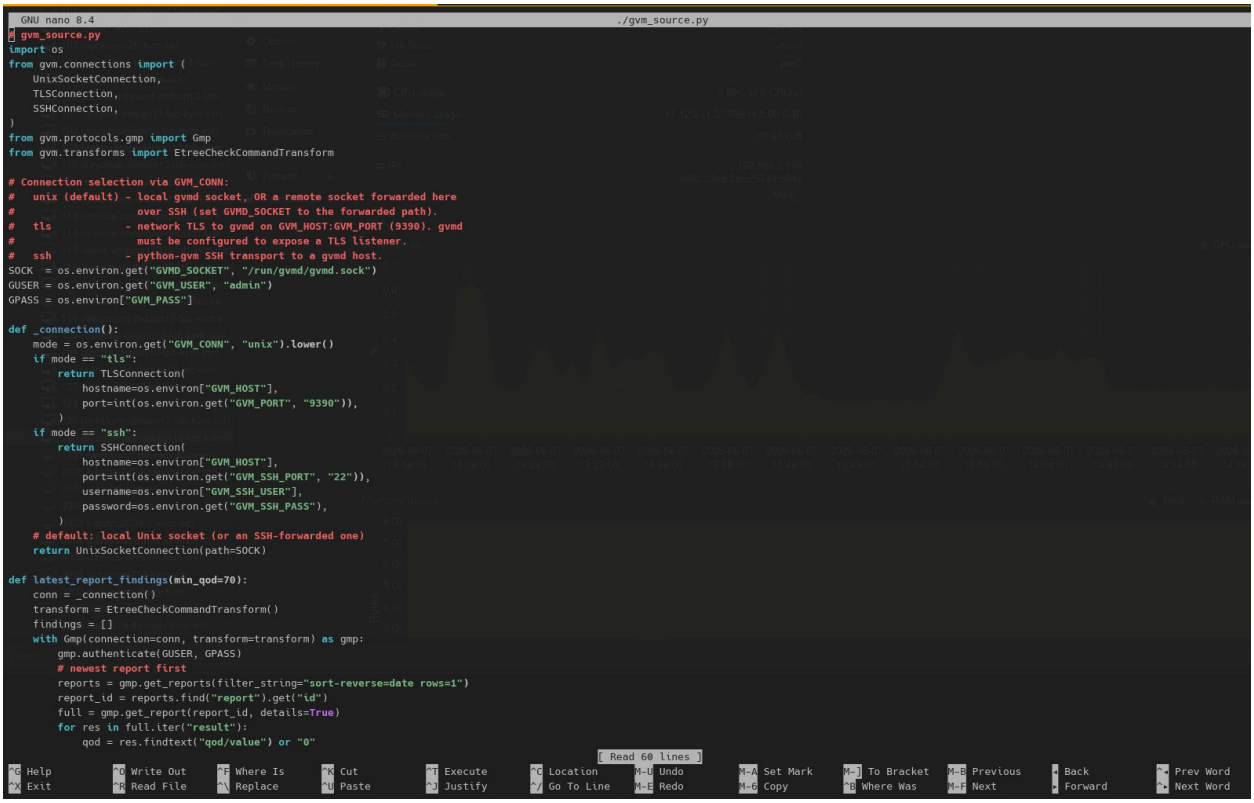

Figure 14 – Greenbone source python script

Part 2 — The prioritization prompt

1. Create vuln_prompt.py. The instruction here is different from Lab 1: we are not classifying one item, we are asking for a **plan** across many.

```
# vuln_prompt.py
VULN_SYSTEM = """You are a security engineer writing a remediation plan from
an OpenVAS scan of a small environment. You receive findings for one host.

Do NOT simply restate CVSS scores. Prioritize by realistic risk in a small
enterprise/home-lab: remote code execution and exposed services outrank local
or theoretical issues; an unauthenticated internet-facing flaw outranks a
high-score flaw on an isolated host.
```

```
Respond with ONLY a JSON object (no markdown fences):
{
  "host": "the host",
  "overall_risk": "critical|high|medium|low",
  "fix_order": [
     {"finding": "name",
      "why_it_matters_here": "one sentence in context",
      "action": "concrete remediation step",
      "effort": "low|medium|high"}
  ],
  "quick_wins": ["the 1-3 highest-value, lowest-effort fixes"]
}"""
```

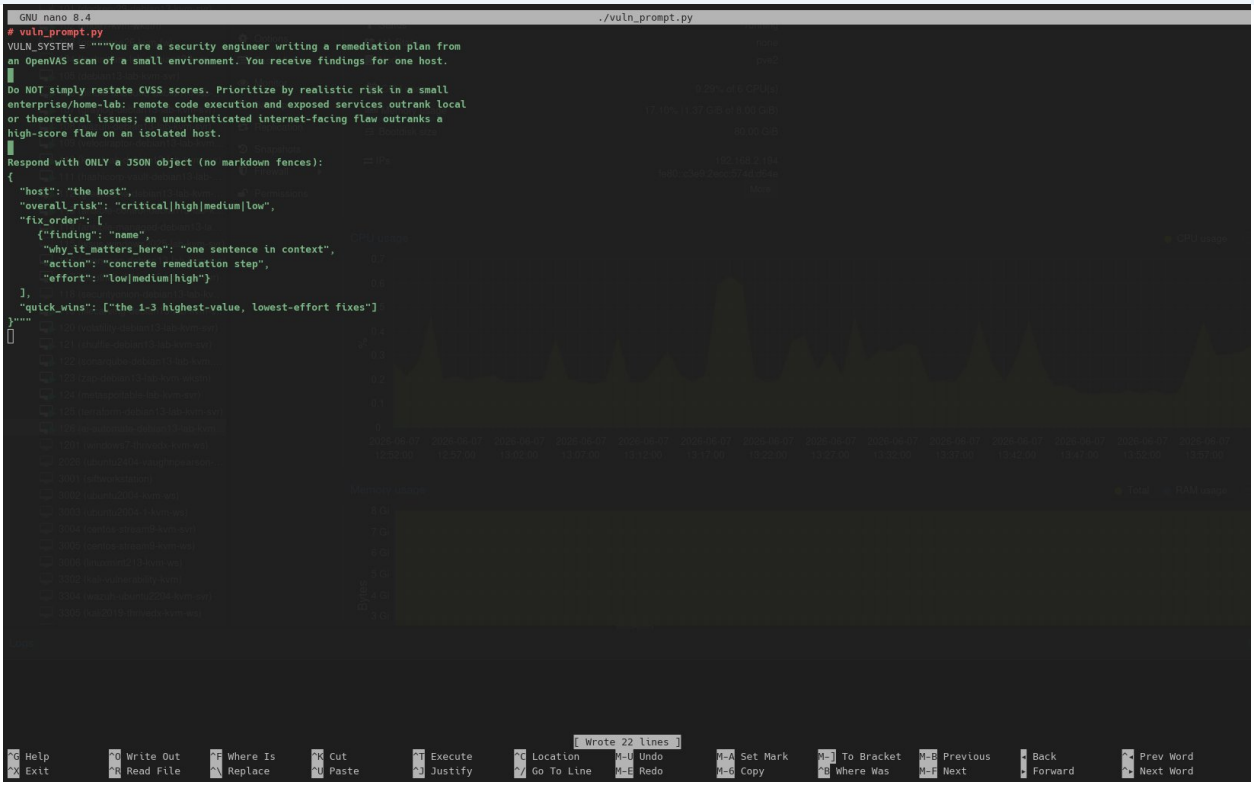

Figure 15 – Vulnerability prompt

Part 3 — The agent

1. Create vuln_agent.py. It groups findings by host and asks Claude for one plan per host, using a stronger model for the denser reasoning.

```
# vuln_agent.py
```

```
import json, sys, collections
import anthropic
from vuln_prompt import VULN_SYSTEM
from gvm_source import latest_report_findings
from siem_source import scrub      # reuse the Lab 1 redaction pass

MODEL = "claude-sonnet-4-6"   # denser reasoning than Lab 1
client = anthropic.Anthropic()

def plan_for_host(host, findings):
    # Scan descriptions can echo detected credentials; redact before sending.
    findings = [{k: scrub(v) for k, v in f.items()} for f in findings]
    resp = client.messages.create(
        model=MODEL,
        max_tokens=1500,
        system=VULN_SYSTEM,
        messages=[{"role": "user",
                   "content": f"Host: {host}\nFindings:\n" +
json.dumps(findings, indent=2)}],
    )
    raw = resp.content[0].text.strip()
    start, end = raw.find("{"), raw.rfind("}")
    return json.loads(raw[start:end + 1])

def main():
    findings = latest_report_findings()
    by_host = collections.defaultdict(list)
    for f in findings:
        by_host[f["host"]].append(f)
    print(f"{len(findings)} findings across {len(by_host)} hosts",
file=sys.stderr)
    plans = []
    for host, fs in by_host.items():
        try:
            plans.append(plan_for_host(host, fs))
```

```
        except Exception as e:
            print(f"  skip {host}: {e}", file=sys.stderr)
    order = {"critical": 0, "high": 1, "medium": 2, "low": 3}
    plans.sort(key=lambda p: order.get(p.get("overall_risk", "low"), 9))
    json.dump(plans, open("vuln_plan.json", "w"), indent=2)
    print(f"Wrote vuln_plan.json with {len(plans)} host plans")

if __name__ == "__main__":
    main()
```

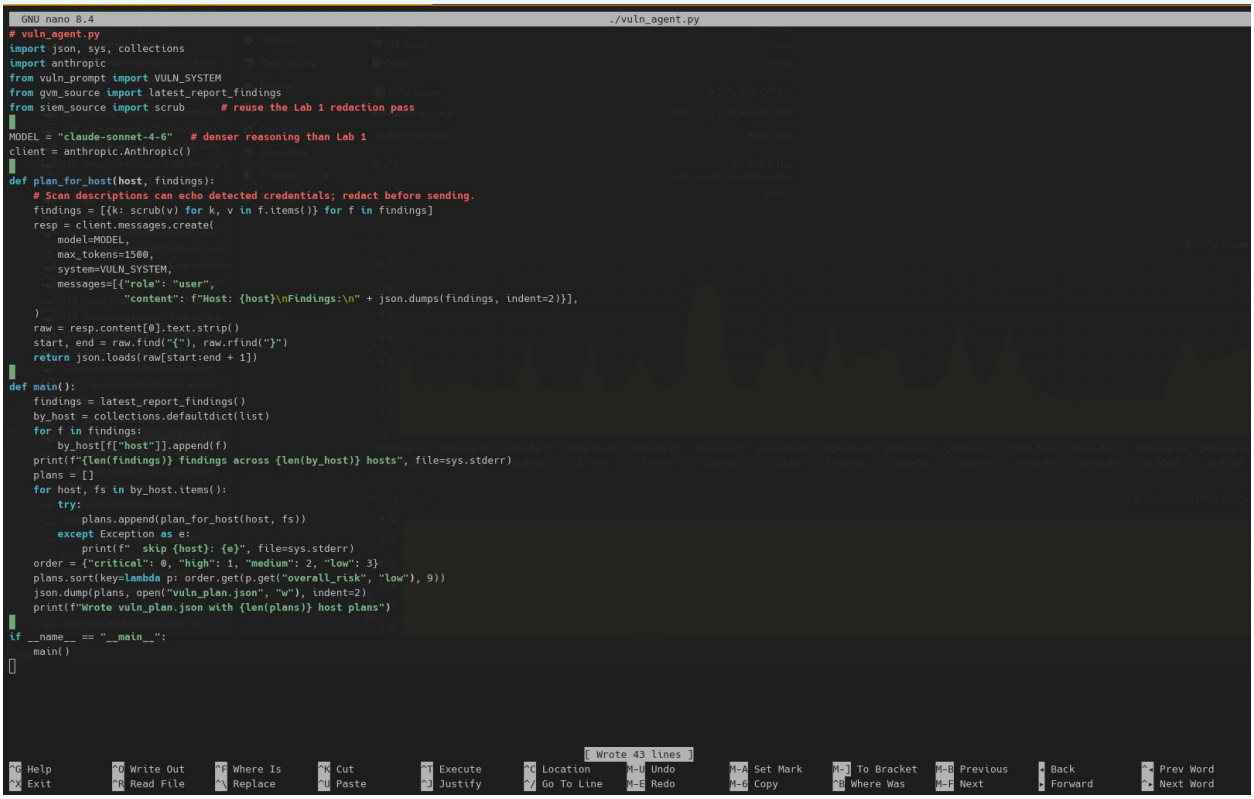

Figure 16 – Vulnerability agent

2. Run it

```
export GVM_PASS="...your-gvm-admin-password..."
# ensure your user can read the gvmd socket (group _gvm on Debian),
# or run via sudo -g _gvm

# Running from a separate host (agent box is not the Greenbone server):
# forward the remote gvmd socket over SSH, then point GVMD_SOCKET at it:
ssh -nNT -o StreamLocalBindUnlink=yes \
```

```
 -L /tmp/gvmd.sock:/run/gvmd/gvmd.sock <ssh-user>@<greenbone-host>

# export GVMD_SOCKET="/tmp/gvmd.sock"   # SSH user must be in _gvm on the
server

# -- or, if gvmd exposes TLS on 9390, skip the tunnel and use:

# export GVM_CONN=tls GVM_HOST=<greenbone-host> GVM_PORT=9390

python vuln_agent.py
```

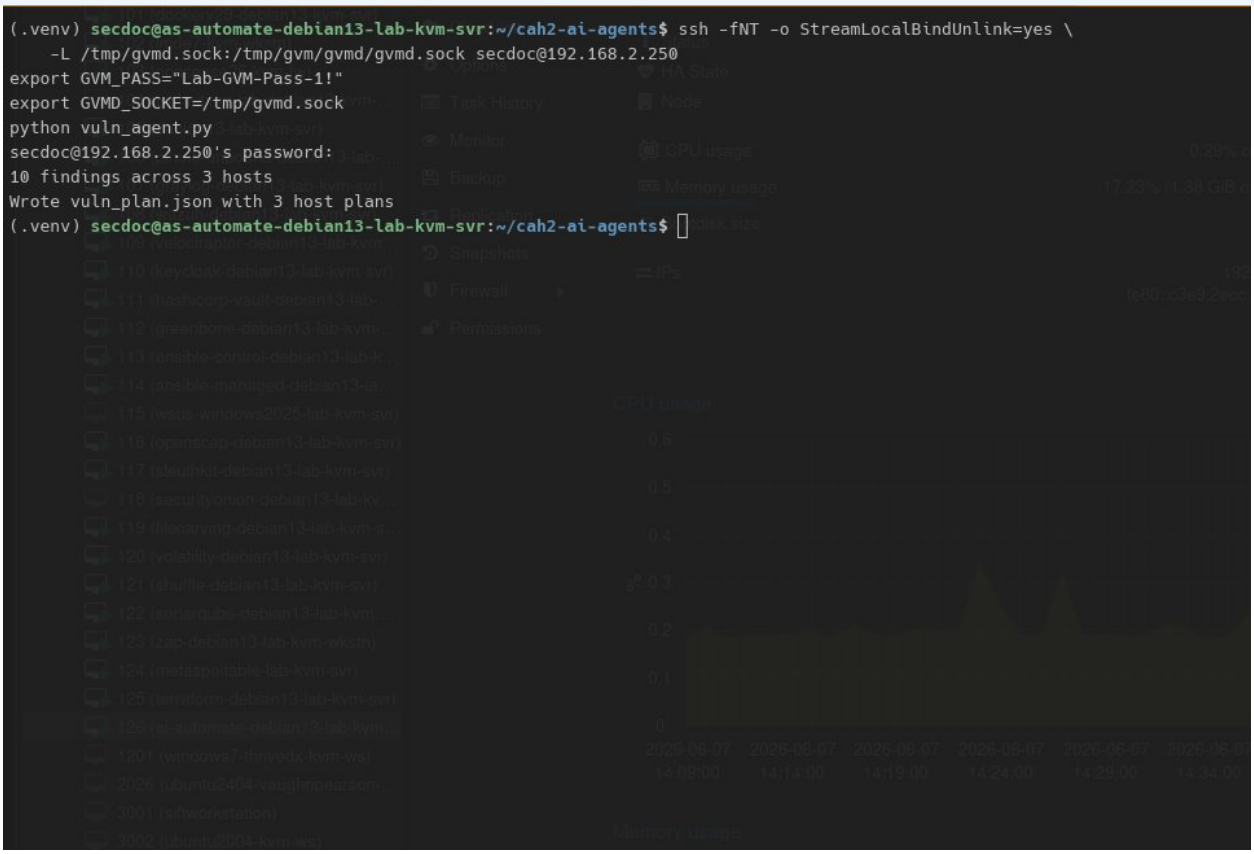

Figure 17 – Successful vulnerability findings pull

Running against the Greenbone Community Containers (Docker)

1. If your Greenbone is the Docker-based Community (as deployed in Domain Category 5 lab) Containers rather than a packaged install, gvmd runs inside a container and its Unix socket lives in a Docker volume — it never appears on the host at /run/gvmd/gvmd.sock, and there is no _gvm group on the host. To reach GMP from outside the container, bind-mount the socket directory to a host path, then forward that host path.

```
# On the Greenbone host, in compose.yaml, for the gvmd AND gsad services

# replace the shared socket volume with a host bind mount:

#     - gvmd_socket_vol:/run/gvmd      (remove this line)

#     - /tmp/gvm/gvmd:/run/gvmd        (add this line)

# and give gvmd a readable socket by adding to its command: --listen-mode=666
```

```
mkdir -p /tmp/gvm/gvmd

docker compose -f compose.yaml down

docker compose -f compose.yaml up -d

ls -l /tmp/gvm/gvmd/gvmd.sock      # the socket is now present on the host
```

2. Then point the SSH forward at the new host path — note /tmp/gvm/gvmd/gvmd.sock, not /run/gvmd/gvmd.sock:

```
ssh -fNT -o StreamLocalBindUnlink=yes \

    -L /tmp/gvmd.sock:/tmp/gvm/gvmd/gvmd.sock <ssh-user>@<greenbone-host>

export GVMD_SOCKET="/tmp/gvmd.sock"
```

3. To confirm gvmd is healthy independently of the host exposure, the stack's bundled gvm-tools container already has the socket mounted:

```
docker compose exec -T gvm-tools \

  gvm-cli --gmp-username admin --gmp-password "$GVM_PASS" socket --xml
"<get_version/>"
```

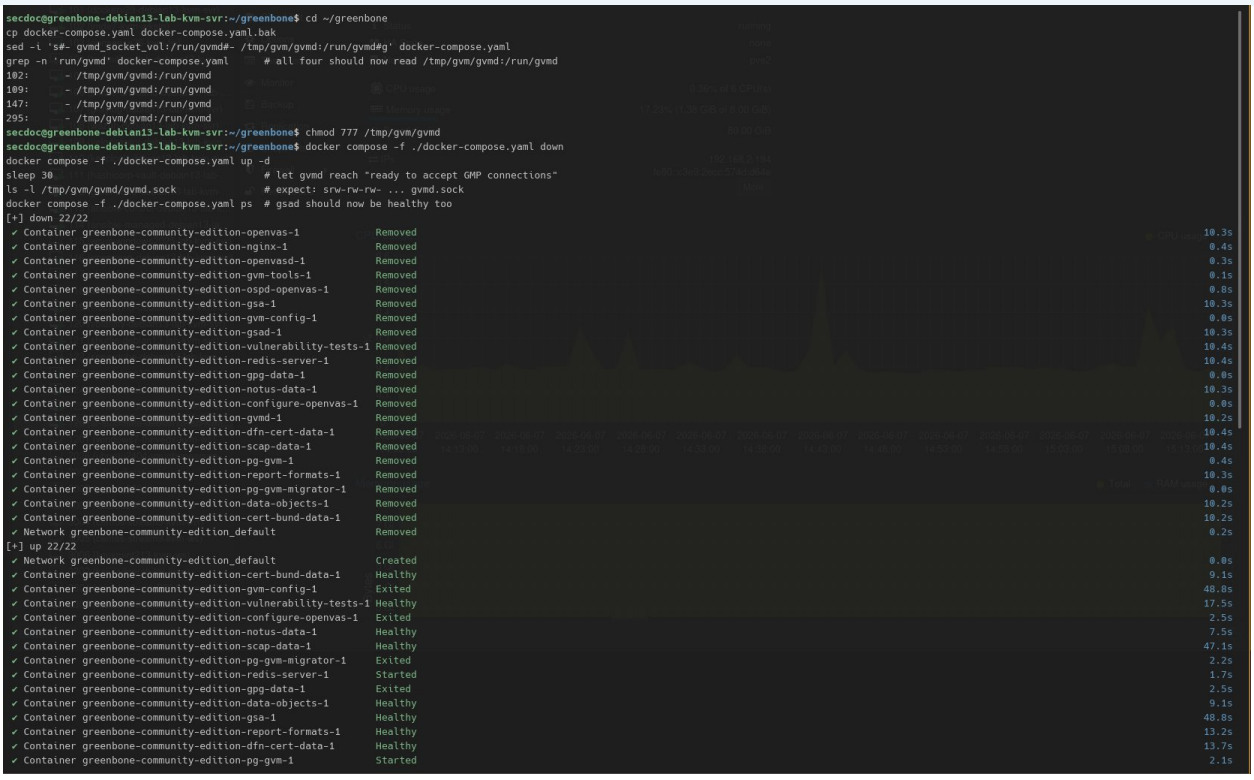

Figure 18 – Adjusted greeenbone docker settings and restart

Part 4 — Render the remediation plan

1. Reuse the rendering pattern from Lab 1. A short script turns vuln_plan.json into a per-host Markdown plan with quick wins called out first — the artifact you would actually hand to whoever owns the host.

```
# render_plan.py
import json

plans = json.load(open("vuln_plan.json"))
out = ["# Vulnerability Remediation Plan", ""]
for p in plans:
    host = p.get("host", "unknown")
    risk = (p.get("overall_risk") or "unknown").upper()
    out.append(f"## {host} — overall risk: {risk}")
    if p.get("quick_wins"):
        out.append("**Quick wins:** " + "; ".join(p["quick_wins"]))
    out.append("")
    for i, step in enumerate(p.get("fix_order", []), 1):
        finding = step.get("finding", "(unnamed finding)")
        effort = step.get("effort", "unknown")
        out.append(f"{i}. **{finding}** ({effort} effort)")
        out.append(f"   - Why here: {step.get('why_it_matters_here', '')}")
        out.append(f"   - Action: {step.get('action', '')}")
    out.append("")
open("vuln_plan.md", "w").write("\n".join(out))
```

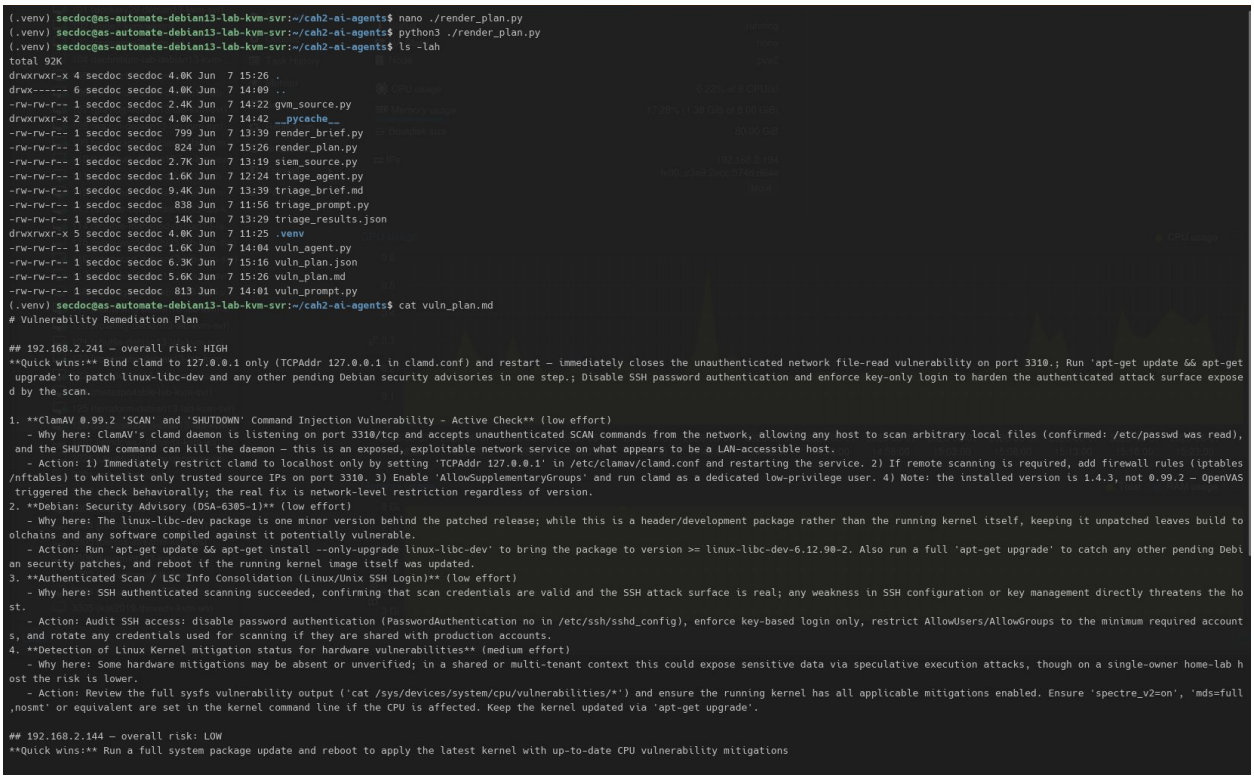

Figure 19 – Vulnerability plan

Validate the Lab

- vuln_agent.py connects over GMP and reports the finding/host counts
- Low-QoD findings (< 70) are excluded — compare the count against the Greenbone web UI
- Each host plan has a fix_order and at least one quick win where applicable
- The ordering reflects context, not just CVSS (e.g., an exposed service ranks above a higher-scored local issue)
- vuln_plan.md reads like a plan a human could execute, not a CVSS dump

Troubleshooting

Symptom	Likely cause and fix
PermissionError on the socket	Your user is not in the _gvm group. Add it (then re-login) or run with sudo -g _gvm.
GvmError: Socket /run/gvmd/gvmd.sock does not exist	gvmd is not reachable on this machine. If the agent runs on the Greenbone host, start gvmd (systemctl start gvmd). If it runs on a separate box, a Unix socket cannot cross the network — forward the remote socket over SSH and set GVMD_SOCKET to the forwarded path, or set GVM_CONN=tls (or ssh) with GVM_HOST. See "Running

Symptom	Likely cause and fix
	from a separate host" under Run it.
No reports found	The scan task has not finished, or no task has ever run. Confirm a completed report exists in the Greenbone web UI first.
Token/length errors from the API	A host has a very large number of findings. Lower the truncation length in gvm_source.py or cap findings per host before sending.
Plans feel generic	Add environment context to the prompt — which hosts are internet-facing, which hold sensitive data. The model prioritizes far better with that context.

Table 5 - Troubleshooting

Lab 3 — The Triage Orchestrator: Cross-Correlating Signals

The two agents are useful alone, but the real value of automation is **correlation** — connecting a SIEM alert to a known vulnerability on the same host. An alert about suspicious activity on a host that Lab 2 flagged as critically vulnerable is a very different priority than the same alert on a hardened host. This lab builds a thin orchestrator that runs both agents, joins their output by host, and asks Claude for a single daily brief.

Objectives

- Run both agents and join their results on the host field.
- Ask Claude to synthesize a single executive-style daily security brief.
- Schedule the orchestrator to run unattended and drop a brief in your inbox or a file.

Prerequisites

- Labs 1 and 2 both **completed and working**. The orchestrator imports triage_agent and vuln_agent and runs each, so fetch_alerts and latest_report_findings must already succeed on their own.
- The shared virtual environment active, with anthropic, python-gvm, and requests installed.
- Credentials and connection settings for both sources, exported in the environment the job runs in: your SIEM variables (WAZUH_* or GRAYLOG_*) and the full GVM connection — GVM_PASS plus GVM_CONN and either GVMD_SOCKET or GVM_HOST/GVM_PORT. GVM_PASS alone is no longer sufficient now that the scanner connection is selectable (see Lab 2).

- For Part B (unattended runs): a Debian 13 host with **systemd user services**. To let the timer fire while you are logged out, enable lingering for your user: loginctl enable-linger $USER.
- Ideally at least one host that appears in both a SIEM alert and a scan finding, so the correlation has something to join and the brief has a lead item.

Part 1 — Join the two signals

1. Create orchestrator.py. It imports the two agents' main functions, runs them, and builds a per-host correlation object.

```
# orchestrator.py
import json, collections, datetime
import anthropic
import triage_agent, vuln_agent

MODEL = "claude-sonnet-4-6"   # or claude-opus-4-8 for the richest synthesis
client = anthropic.Anthropic()

BRIEF_SYSTEM = """You are the lead analyst writing the morning security brief
for a small environment. You receive (1) triaged SIEM alerts and (2) a
vulnerability remediation plan, already joined by host. Write a concise brief:
lead with anything where an active alert coincides with a known vulnerability
on the same host — those are the day's priorities. Then note other notable
items. Be direct and skimmable. Plain prose, short paragraphs, no fluff."""

def correlate():
    alerts = triage_agent.main(20)
    vuln_agent.main()
    plans = json.load(open("vuln_plan.json"))
    by_host = collections.defaultdict(lambda: {"alerts": [], "vuln": None})
    for a in alerts:
        # Join key: the Wazuh source supplies "host" (agent name); Graylog
        # supplies "source". For this to line up with the scanner's host,
        # both sides must use the same identifier (hostname OR IP);
        # normalize here (e.g. resolve names to IPs) if they differ.
        host = a["_alert"].get("host") or a["_alert"].get("source") or
"unknown"
        by_host[host]["alerts"].append(
```

```
            {"severity": a["severity"], "summary": a["summary"]})
    for p in plans:
        by_host[p["host"]]["vuln"] = {
            "overall_risk": p["overall_risk"],
            "quick_wins": p.get("quick_wins", [])}
    return dict(by_host)


def write_brief(correlated):
    resp = client.messages.create(
        model=MODEL, max_tokens=2000, system=BRIEF_SYSTEM,
        messages=[{"role": "user",
                   "content": "Joined signals by host:\n" +
json.dumps(correlated, indent=2)}],
    )
    text = resp.content[0].text
    fname = f"daily_brief_{datetime.date.today()}.md"
    open(fname, "w").write(text)
    print(f"Wrote {fname}")
    return fname


if __name__ == "__main__":
    write_brief(correlate())
```

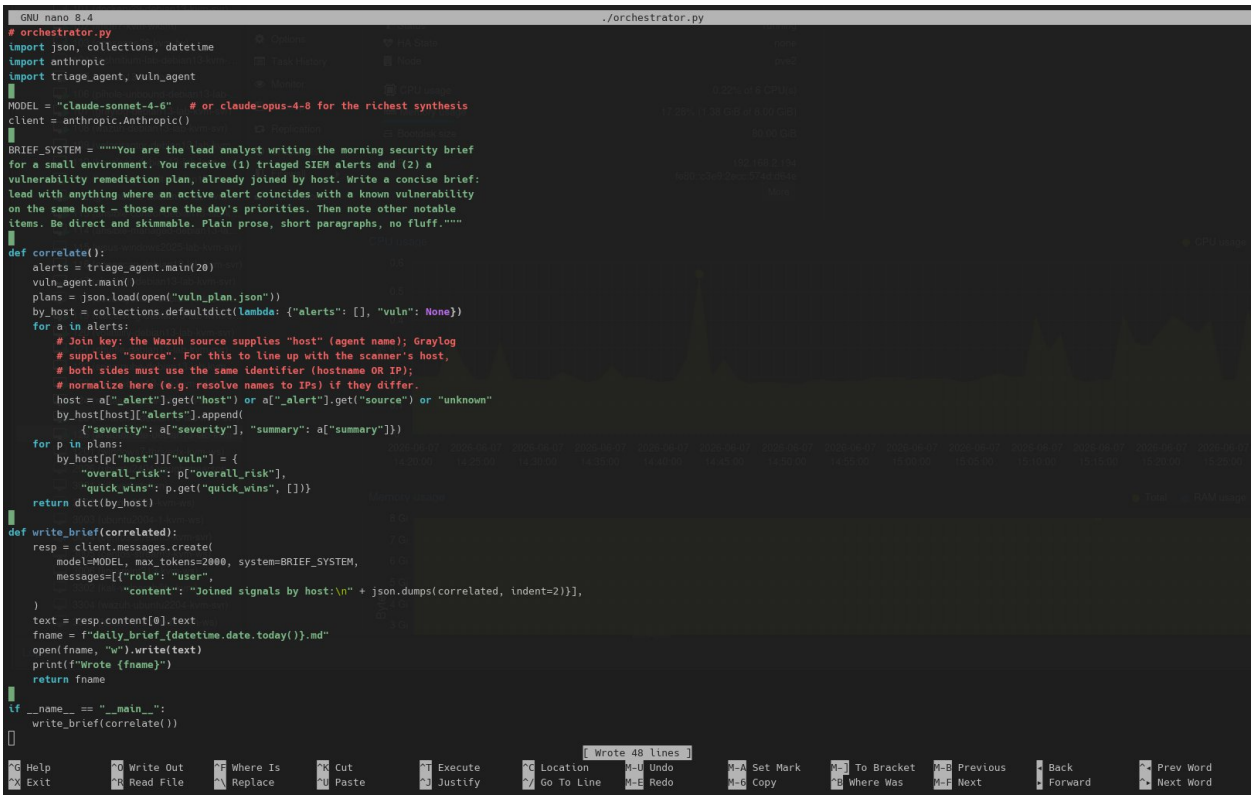

Figure 20 – Orchestration script

This is where the LLM earns its keep

A rule engine could join two tables on a host field. What it cannot do is read “suspicious outbound connection” next to “host has an unpatched remote-code-execution flaw” and write a sentence explaining why those two facts together are the most important thing in your environment today. That synthesis — turning joined data into prioritized narrative — is the judgment you are automating.

Part 2 — Run it unattended

1. Schedule the orchestrator with a systemd timer (preferred on Debian 13) or cron. The wrapper sources your secrets, activates the venv, and runs the job.

```
# ~/cah2-ai-agents/run_brief.sh

#!/usr/bin/env bash

set -euo pipefail

source ~/.cah2-secrets

export WAZUH_PASS GRAYLOG_TOKEN GVM_PASS GVM_CONN GVMD_SOCKET GVM_HOST
GVM_PORT  # whatever your sources need

cd ~/cah2-ai-agents

source .venv/bin/activate
```

```
python orchestrator.py
```

Figure 21 – Run orchestration

```
chmod +x ~/cah2-ai-agents/run_brief.sh
```

Figure 22 – Make executable

```
# ~/.config/systemd/user/security-brief.service

[Unit]

Description=CAH2 daily security brief


[Service]

Type=oneshot

ExecStart=%h/cah2-ai-agents/run_brief.sh
```

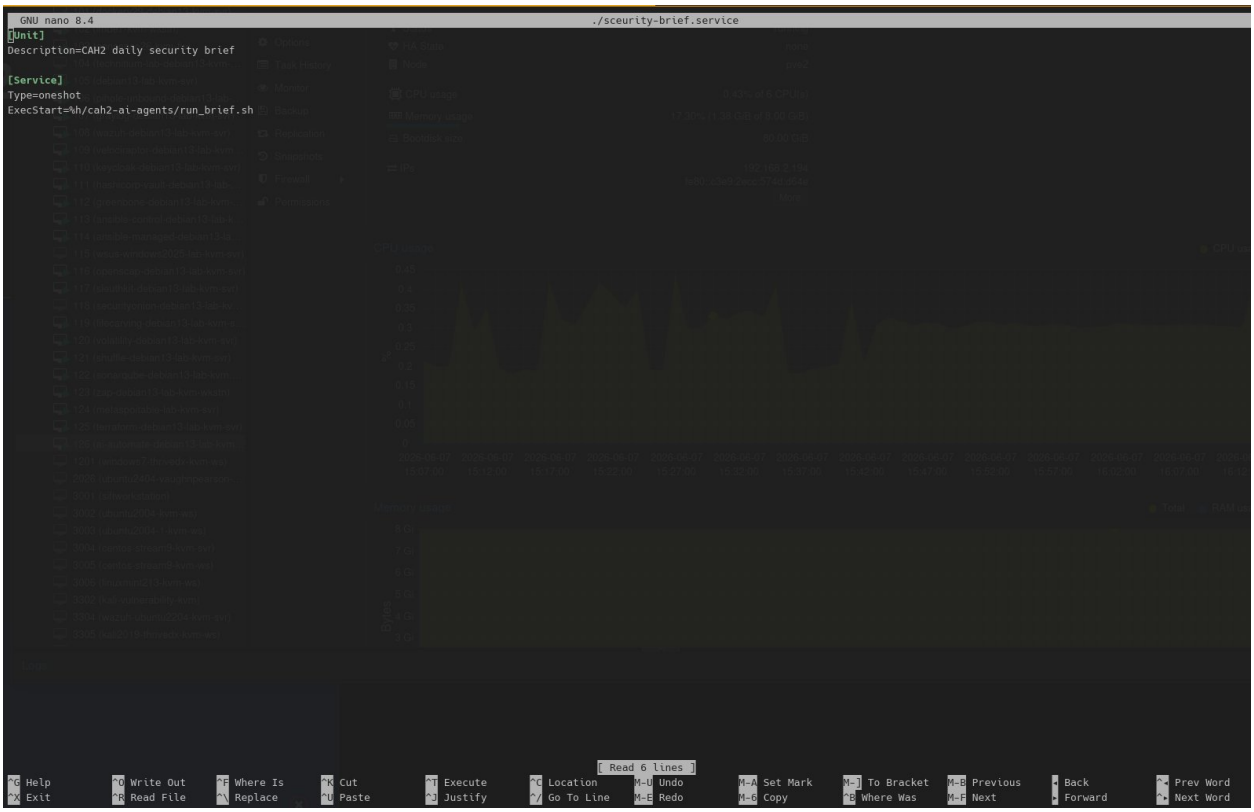

Figure 23 – Create service file

```
# ~/.config/systemd/user/security-brief.timer
[Unit]
Description=Run CAH2 security brief each morning

[Timer]
OnCalendar=*-*-* 07:30:00
Persistent=true

[Install]
WantedBy=timers.target
systemctl --user daemon-reload
systemctl --user enable --now security-brief.timer
systemctl --user list-timers security-brief.timer
```

Figure 24 – Create timer

```
mkdir -p ~/.config/systemd/user
#     -> ~/.config/systemd/user/security-brief.service
#     -> ~/.config/systemd/user/security-brief.timer


# 3. then register and start the timer (these are commands, not file content)
systemctl --user daemon-reload
systemctl --user enable --now security-brief.timer
systemctl --user list-timers security-brief.timer
```

Storing secrets for an unattended job

An unattended job cannot type a password. The ~/.cah2-secrets file at chmod 600 is the minimum bar. For anything beyond a lab, graduate to a real secrets manager (systemd LoadCredential, HashiCorp Vault, or your OS keyring) so the key is never sitting in a plaintext file at all.

Figure 25 – Initialize service and timer

Validate the Lab

- orchestrator.py runs both agents and writes a dated brief file
- A host that has both an alert and a vulnerability is called out first in the brief
- The brief reads as prose a manager could skim, not as raw JSON
- The systemd timer is listed and shows a next-run time
- A manual run via run_brief.sh succeeds with the same output as running by hand

Where to Go Next

You have built a read-only triage pipeline. The natural extensions deepen either the intelligence or the integration, and each maps to a tool you already met in this supplement:

- **Add tool use (function calling).** Instead of you pasting context, let Claude call functions you define — “look up this host’s role,” “fetch the last 24h of alerts for this IP.” The Messages API supports tool use directly; the model decides when to call your functions and you keep full control of what they do.
- **Feed the brief into TheHive or your ticketing system.** Turn the orchestrator’s output into cases automatically, closing the loop from detection to tracked remediation.
- **Correlate across time.** Persist each day’s brief and let the agent compare today against the trailing week — “this is the third day this host has alerted.”
- **Run a local model for air-gapped labs.** If your environment cannot reach the API, the same agent structure works against a locally hosted open-weight model. The prompts and parsing carry over unchanged; only the client call differs.
- **Harden the safety posture.** Add an allowlist of fields that may be sent to the API, log every request for audit, and add a human-approval gate before any future write-action is ever introduced.

The discipline that matters most

It is tempting to let an agent act — quarantine a host, close a ticket, re-run a scan. Resist that until you have lived with its judgment for a long time and instrumented it heavily. Every lab here is read-only by design: the agent informs, the human decides. That boundary is not a limitation of the technology; it is the responsible default for putting a probabilistic system anywhere near your security operations.